Reviewer Pack — Minimal Rank of Quasi-Orthogonal Matrices over Function Fiel…

Entry 57639c301858 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 06:28:51 UTC

Minimal Rank of Quasi-Orthogonal Matrices over Function Fields vs AC0 Parity Circuit Size

Entry ID: `57639c301858` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-24 06:28:51 UTC

1. Conjecture

Field A (mathematical branch): Linear Algebra (Quasi-Orthogonal Matrices) **Field B** (complexity object): Complexity Theory: AC0 Parity Complexity

Statement:

> ['For every function field K with q elements, there exists a constant c > 0 such that for any Boolean function f over K with parity complexity p(f) ≤ n, the minimal rank of a quasi-orthogonal matrix system representing f is at least cn.', 'Equivalently, for all k-CNF formulas F with n variables and clause density d(F) ≤ 1/q, there exists a constant c > 0 such that the minimal rank of the matrix A_F representing the negated clauses of F is at least $c \cdot n^{(2/3)d(F)}(-1/3)$.', 'If this conjecture holds, then for any AC0 parity circuit C with size s and depth d(C) ≤ n^(2/3), there exists a function field K with q elements such that the minimal rank of the matrix representing the negated clauses of C is at least $c \cdot q^{(1/3)d(C)}(-2/3)$.']

Rationale (proposer's reasoning):

> ['Quasi-orthogonal matrices have been studied in linear algebra and their properties could provide a new perspective on circuit complexity. The minimal rank of these matrices might relate to the size of AC0 parity circuits, offering insights into the limitations of AC0 computation.', 'The constructive mapping from function fields to quasi-orthogonal matrices allows us to explore the relationship between arithmetic structures in algebraic geometry and computational complexity.', 'This conjecture aims to link a classical algebraic structure with a specific circuit complexity measure, which may reveal underlying structural relationships that are not apparent through traditional methods.']

Taxonomy category: `linear_algebra_to_circuit_complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `49c8506b802ff623`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a given AC0 parity circuit C, if the minimal rank r of the matrix representing its negated clauses is at least c·q^(1/3)d(C)(-2/3), where q is the number of elements in the function field K and d(C) ≤ n^(2/3), then the conjecture is supported. If any seed produces a metric r < c·q^(1/3)d(C)(-2/3), the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	SAFE
NATURAL_PROOFS	SAFE	1.00	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Minimal rank quasi-orthogonal matrices AND AC0 parity complexity` - `Function fields AND minimal rank representation quasi-orthogonal matrices AND AC0 circuits` - `Quasi-orthogonal matrix systems AND AC0 circuit size lower bound function fields`

Top relevant hits considered: - [http://arxiv.org/abs/2006.02374v2] On tensor rank and commuting matrices - [http://arxiv.org/abs/1501.01773v1] Estimates for the growth of inverse determinant sums of quasi-orthogonal and number field lattices - [http://arxiv.org/abs/2403.03513v3] Spectrum of random centrosymmetric matrices; CLT and Circular law - [http://arxiv.org/abs/0711.0795v4] Finite-Dimensional Representations of Hyper Loop Algebras Over Non-Algebraically Closed Fields - [http://arxiv.org/abs/math/0402188v5] Structures and Representations of Generalized Path Algebras - [http://arxiv.org/abs/2508.04609v2] Analog Solver Circuit for Linear Symmetric Positive-Definite Systems at a Complexity Independent of Matrix Size - [http://arxiv.org/abs/1210.1461v1] A Scalable CUR Matrix Decomposition Algorithm: Lower Time Complexity and Tighter Bound - [http://arxiv.org/abs/2011.02044v5] Circuit lower bounds for low-energy states of quantum code Hamiltonians

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
from fractions import Fraction
import random
import math

def generate_k_cnf(n, q):
    clauses = []
    for _ in range(q):
        clause = set()
        while len(clause) < 2:
            literal = random.randint(1, n)
            if literal not in clause and -literal not in clause:
                clause.add(literal)
        clauses.append(clause)
```

```
    return clauses

def construct_matrix(clauses, q):
    n = len(clauses[0])
    matrix = [[Fraction(0) for _ in range(n + 1)] for _ in range(q)]
    for i, clause in enumerate(clauses):
        for literal in clause:
            if literal > 0:
                matrix[i][literal] += Fraction(1)
            else:
                matrix[i][-literal] -= Fraction(1)
    return matrix

def min_rank(matrix):
    m = len(matrix)
    n = len(matrix[0])
    rank = 0
    for i in range(m):
        if all(matrix[j][i] == 0 for j in range(i, m)):
            continue
        pivot_row = i
        while matrix[pivot_row][i] == 0:
            pivot_row += 1
            if pivot_row == m:
                return rank
        matrix[i], matrix[pivot_row] = matrix[pivot_row], matrix[i]
        for j in range(m):
            if i != j and matrix[j][i] != 0:
                factor = -matrix[j][i] / matrix[i][i]
                for k in range(n + 1):
                    matrix[j][k] += factor * matrix[i][k]
        rank += 1
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    q = random.randint(1, min(n, 20))
    clauses = generate_k_cnf(q, n)
    matrix = construct_matrix(clauses, q)
    rank = min_rank(matrix)
    p_f = len(clauses) / q
    d_F = q / n
    c = 0.5  # Hypothetical constant for the sake of testing
    expected_rank = c * q**(1/3) * d_F**(-2/3)
    conjecture_holds = rank >= expected_rank
    counterexample = "" if conjecture_holds else f"rank={rank}, expected={expected_rank}"
    return {
        "metric_name": "minimal_rank",
        "metric_value": rank,
        "instances_tested": q,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
```

```python
    seeds = [int(x) for x in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    std_rank = math.sqrt(sum((r["metric_value"] - mean_rank)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fracti
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"rank too low\" first_failing_seed={first_failin
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2d1538f1.py", line 86, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2d1538f1.py", line 66, in run_trial
    matrix = construct_matrix(clauses, q)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2d1538f1.py", line 35, in construct_mat
    matrix[i][literal] += Fraction(1)
    ~~~~~~~~~^^^^^^^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed before producing any data, which means that we cannot determine whether the conjecture is supported or falsified based on the pre-registered criteria. | next: Re-run the test with proper error handling to ensure it completes without crashing and produces results for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12743
2	propose	ollama_remote	glm4:latest	0	0	12139
3	preregistration	ollama_remote	glm4:latest	0	0	6307
4	novelty	ollama_remote	glm4:latest	0	0	4686
5	novelty	ollama_remote	glm4:latest	0	0	10802
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15498
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10381
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9464
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10076
10	judge	ollama_remote	glm4:latest	0	0	8259

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 100355 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/57639c301858.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/57639c301858.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/57639c301858.tar.gz` (if generated)